

```
exercise_4.py x
1  ## Course 1: Natural Language Processing
2  ## Module 1: Intro to NLP in Python
3  ## Coding Exercise 4
4  from nltk.tokenize import word_tokenize
5  from nltk import pos_tag, ne_chunk
6
7  def find_names(text):
8      tags = pos_tag(word_tokenize(text))
9      labeled_chunks = ne_chunk(tags, binary=True)
10     return set(
11         "".join(word[0] for word in chunked_word)
12         for chunked_word in labeled_chunks
13         if hasattr(chunked_word, "label") and chunked_word.label() == "NE"
14     )
15
16 text = "The various continuations of William of Tyre above mentioned represent the op
print(find_names(text))
```

Guide x

Collapse

<

>

⚙

☰

Coding Exercise 4

Problem

Create the function `find_names` that takes a string and returns a set of all names used in the string.

Expected Output

After creating the `find_names` function, pass it the following variable and print the return value.

```
text = "The various continuations of William of Tyre above mentioned represent the
```

Click the `TRY IT` button to run the file and check your work. The expected output is shown below. **Note**, sets are unordered so your result may be in a different order. You should see the same elements in the set.

TRY IT

```
{'Nicetas', 'Eastern', 'William', 'Cinnamus', 'Anna',
'Richard', 'Professor Bury', 'Tyre', 'Franks'}
```

```
, 'Cinnamus', 'Nicetas', 'William', 'Professor Bury', 'Richard', 'Eastern', 'Franks'}
```

Submit your work for evaluation.

Here is one possible solution:

```
from nltk import pos_tag, ne_chunk
from nltk.tokenize import word_tokenize

text = "The various continuations of William of Tyre above mentioned rep

def find_names(data):
    tags = pos_tag(word_tokenize(data))
    labeled_chunks = ne_chunk(tags, binary=True)
    return set(
        " ".join(word[0] for word in chunked_word)
        for chunked_word in labeled_chunks
        if hasattr(chunked_word, "label") and chunked_word.label() == "NE"
    )

print(find_names(text))
```

- Import `pos_tag`, `ne_chunk`, and `word_tokenize` from the `nltk` module
- Create the `find_names` function that takes a string
- Tokenize the string and apply the parts of speech tags
- Use the named entity chunker to chunk the tokens
- Find all of the chunked words with the label `NE` and add them to a set
- Return the set

Check It!

LAST RUN on 12/27/2023, 3:40:04 AM

```
{'Cinnamus', 'Anna', 'Franks',
'William', 'Professor Bury',
'Richard', 'Eastern', 'Tyre',
'Nicetas'}
test_1 (test_exercise_4.test_ex4) ...
ok
test_2 (test_exercise_4.test_ex4) ...
ok
test_3 (test_exercise_4.test_ex4) ...
ok
```

Next